MemoryCircle - TA review guide

This is a rehearsal guide, separate from the report submitted to D2L. Use the finalized main branch based on the supplied Expo app. The old Express prototype and its 16 tests are not this demonstration. This version has 24 repository tests.

1. Prepare together before the lab

1. Put the final source on the presentation laptop. Open the repository on branch main and the revised report. Confirm the TA can access the repository.
2. Install Node.js 24, including npm. Open a terminal in the directory containing package.json. Run:
npm ci
npm test
npm run typecheck
npm run lint
npm run build:web
3. Start with npm run web. Open the localhost address Expo prints. For a native demonstration, start with npm start and use an SDK 58 compatible client/development build already tested on your phone. Do not spend the evaluation installing a new client.
4. Use the same browser profile/device throughout the persistence demonstration. Choose a presentation profile with enough free storage and fewer than 27 photos already saved this UTC month.
5. Have demo-assets/photo-1.jpg, photo-2.jpg, photo-3.jpg and demo-tone.mp3 available in the laptop/phone picker. The files are synthetic and contain no personal photos or third-party music.
6. Keep these files open in editor tabs: src/app/create-memory.tsx, src/app/memory/[id].tsx, src/data/memory-repository.ts, src/data/photo-store.native.ts, src/data/photo-store.web.ts, tests/memory-repository.test.mjs and docs/acceptance-and-backlog.md.
7. Rehearse each handoff. Every member should know the whole workflow, not only their speaking part. Do not clear storage or run reset-project during the evaluation.

Suggested split: Akande introduces and closes; Nimo demonstrates creation/photos; Seriki explains persistence/architecture; Uba demonstrates layouts/reel/audio and automated tests. This is a speaking plan; each person should describe the contribution they actually completed.

2. Opening - Akande (about 45 seconds)

Show: app Home screen, then the report's implementation summary.
Say: "Our project is MemoryCircle. This increment is an Expo and React Native app for saving memories on one device. We can create a memory with a title, story and photos, reopen it, choose a collage layout, and play a photo slideshow with selected audio. We focused on making that complete local workflow reliable and testable."
Say: "P1 proposed shared private groups and a cloud backend. We changed the initial implementation to an Expo local prototype. Accounts, shared groups, invitations and automatic video exports are still backlog work. The Your circle card is a future-feature message."
Point to the working recent-memory list/empty state. Do not click the avatar or promise a login flow: it is not authentication.
Handoff: "Nimo will show the create-and-save workflow."

3. Create a memory - Nimo (about 2 minutes)

Action 1: press Create a memory. Show the title/story inputs and photo picker.
Say: "This screen is create-memory.tsx. The title is required and limited to 80 characters. The story is required and limited to 2000. Photos are optional."
Action 2: leave title/story empty and press Save memory.
Expected: inline error; no new memory and no navigation away.
Say: "The repository validates before copying photos or writing metadata, so invalid input does not create an incomplete item."
Action 3: enter title "P2 rehearsal" and story "We used three sample photos to test saving, layouts and the photo reel."
Action 4: press Choose Photos. Select the three provided JPEGs. Confirm the picker and point out all three previews.
Say: "We accept JPEG and PNG. The prototype checks ten photos per memory, ten megabytes and twenty megapixels per image, and thirty photos per local device per UTC month. P1's eventual quota is per member and group; this local quota is not that server-side rule."
Action 5: press Save memory once. Wait for Home and the new card.
Expected: one new card at the top. A saving guard prevents repeated taps while the operation is active.
Action 6: open the card. Show title, story, date and photos.
Say: "The memory is now stored locally. Next we will show that it survives reopening."
Handoff: "Seriki will explain where it is stored and the failure handling."

4. Persistence and architecture - Seriki (about 90 seconds)

Action 1: on the detail screen choose Grid. Go back, reopen the same memory and show Grid still selected.
Action 2: refresh the browser, or close/reopen the already rehearsed native app. Open P2 rehearsal again.
Expected: the title, story, three images and chosen layout remain. The selected audio file and current reel position are not saved preferences.
Say: "Screens use one shared repository instead of each editing an AsyncStorage array independently. Metadata uses the memorycircle_memories key. On native, selected images are copied into the app's document directory. On web, we save data URIs because temporary blob URLs disappear after a reload."
Show: architecture diagram in report and repository.ts, memory-repository.ts, then the platform photo-store files.
Say: "The write queue serializes local changes so two overlapping saves do not overwrite the same old list. The memory record is published after all photo copies succeed. If a copy or metadata write fails, we remove copied photos where possible and show an error. If the existing JSON is corrupted, we stop rather than replace it with an empty list."
Say: "These are single-runtime local guarantees. We do not have a server database, account authorization or encrypted group sharing yet. Browser storage quota may also be lower than our logical photo cap."
Handoff: "Uba will demonstrate presentation and testing."

5. Collage, reel and audio - Uba (about 90 seconds)

Action 1: switch Classic -> Grid -> Scrapbook. Use three photos so Classic's large first image is visible. Point out the different layout shapes.
Say: "Collage choices change presentation, not the original title, story or photo ordering. We persist the selected layout."
Action 2: press Play Photo Reel. Watch photo 1 advance after about 2.5 seconds, then watch the index wrap after the last photo.
Say: "The reel is a timed in-app slideshow, not a generated MP4. It loops until Stop Reel. Automatic monthly rendering, job status and downloading are future work."
Action 3: press Stop Reel. Confirm the reel section disappears.
Action 4: press Choose Background Music and select demo-tone.mp3. Then press Play Music.
Expected: the synthetic tone plays. Press Stop Music. Start it again, then navigate back and confirm it pauses.
Say: "Audio is selected with DocumentPicker and played with Expo Audio on native or an HTML audio element on web. Selection is currently for this screen session; it is not yet a saved licensed music catalogue."
If your browser/device blocks audio, use the rehearsed device and a direct user click. Do not describe untested audio as working on every platform.

6. Automated tests and requirement traceability - Uba, with Seriki (about 2 minutes)

Show terminal in the source directory. Run npm test and wait for the final summary.
Expected: 24 pass, 0 fail. These are tests for this Expo prototype; do not show the old 16-test Express output.
Say: "Our tests exercise the same repository used by the screens, with injected storage and file adapters. They cover normal saves, reopen, invalid input, corruption, concurrent writes, photo limits, cleanup after failures, saved layouts and reel index wraparound. OS pickers, native files and audible playback also need manual device checks."
Show tests/memory-repository.test.mjs and src/data/memory-repository.ts side by side.
Trace one story clearly:
- Requirement: P2-M01 says a nonblank title/story should persist and reopen; invalid input should create nothing.
- Implementation: validateDraft, create and list/find in memory-repository.ts.
- Verification: the first persistence test recreates the repository and finds the same record; blank-title/story tests assert no stored value.
Then show one nontrivial failure test: metadata/quota failure removes copied photos, or overlapping saves at 29 photos allow only one more.
Run npm run typecheck and npm run lint; successful commands return without errors. Show the recorded web-export result if the TA asks about the build; a fresh export may take longer than the demo.
Say: "Passing repository tests is evidence for these local rules. It is not evidence that cloud authorization or a video-generation service exists."

7. Backlog, teamwork and closing - Akande / all (about 1 minute)

Show docs/acceptance-and-backlog.md and the report backlog table.
Say: "We kept the original P1 stories and estimates visible. Photo input, browsing and music have partial progress; cloud accounts/groups, membership changes, schedules and generated recaps remain incomplete. We added explicit P2 stories for this local increment instead of marking the larger P1 stories finished."
Each member states a short truthful contribution and points to the relevant file or review evidence. Suggested areas: Akande integration/requirements/report; Seriki repository/persistence/navigation; Nimo photo selection/storage/validation; Uba collage/reel/audio/tests. Do not invent commit authors, dates, hours or a hosting test.
Show the main branch, merged PR #2, commits and changed files. Explain that all four members worked together and reviewed the increment while Akande operated GitHub through osakande-lab, so commits appear under one account. Explain that the friend-provided source is the starting point and the P2 integration fixes/tests are subsequent changes. The original ZIP is preserved locally. Do not claim its entire history appears in the new repository branch.
Closing: "Our next step is the shared-data foundation: accounts, private groups and membership checks. Then we can move local memories to backend storage and add an actual video-generation pipeline. This increment gives us a working local flow and tests to build on."

8. Likely TA questions - short answers

Why Expo instead of the P1 stack? "The team's working implementation was built in Expo. We used that working code and recorded the change. It supports a mobile local flow; the shared backend still needs implementation."
Is Your circle a real group? "No. It explains planned functionality. There is no membership or sharing flow yet."
Where are photos stored? "Native app documents; on web, data URIs alongside metadata. No public cloud URLs. Local storage alone does not provide account-based privacy."
Is the reel a video file? "No. It advances images in memory every 2.5 seconds. There is no encoder or MP4 download."
Can all 30 large photos fit in a browser? "Not necessarily. Browser quota can be reached earlier; the save fails visibly rather than publishing an incomplete memory. Native app storage has different limits."
What happens if one copy fails? "Earlier successful copies are removed where possible; the memory metadata is not committed. Cleanup failure is surfaced, not called a success."
Does the queue solve multi-device races? "No. It serializes writes in one app runtime. Shared cloud storage will need server transactions and authorization."
What if stored data is corrupted? "The app reports the read failure and preserves the existing value. It does not silently reset the user's collection."
Does music persist? "No. Collage preference persists; current audio selection is session-only."
What was actually tested? "24 repository tests, TypeScript, lint and the recorded web build/walkthrough. Native device behavior must be verified on the device used for evaluation."

9. Recovery plan

If the app will not start: confirm the working directory, Node/npm versions and lockfile installation. Show a previously verified build/test log while fixing setup; do not switch to the old Express prototype without explaining it.
If the picker is cancelled: choose again; cancellation must not save an empty or duplicate memory.
If storage is full: keep existing memories; use a prepared clean test profile/device. Do not delete personal data in front of the TA. Explain quota handling and show its automated test.
If the main machine fails: use a second rehearsed device. Screenshots or logs are supporting evidence, not a substitute for claiming a live demo ran.
After the review: update the backlog with TA feedback and submit the revised five-page report through D2L. The rehearsal guide is for the team unless the TA asks for it.